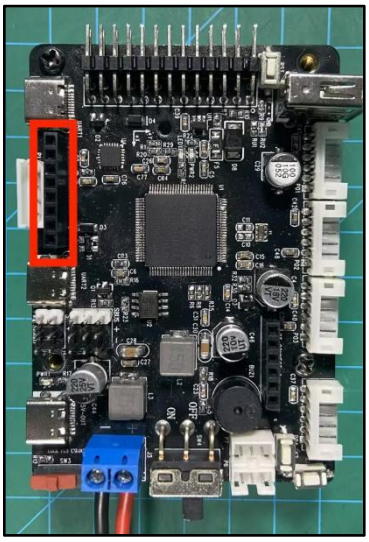
RRC Bluetooth Control Reception and Processing Source Code Analysis

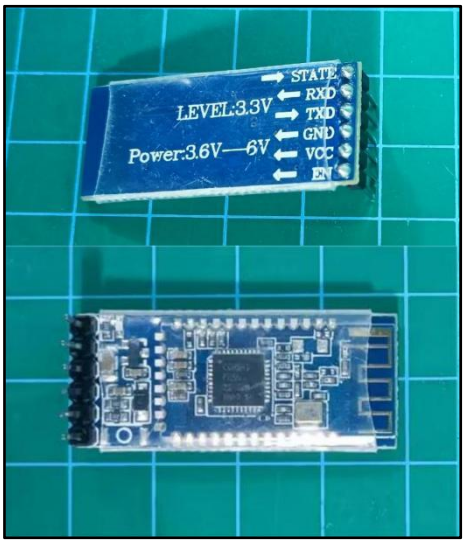
1. Program Logic

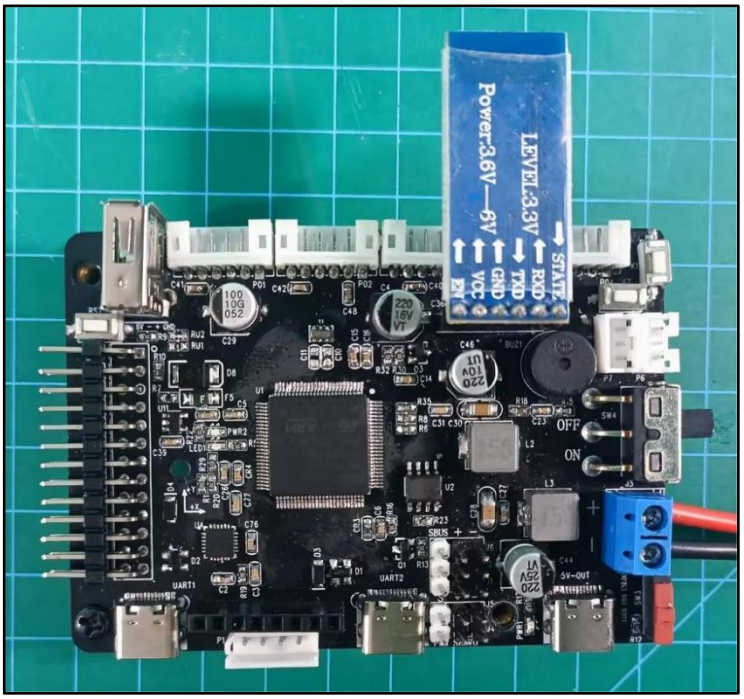
This section mainly covers controlling the car's movement and pan-tilt servo rotation via the app's Bluetooth.

2. Hardware Connection

Correctly insert the Bluetooth module into the car control board. The installation position and orientation are shown in the diagram below:







3. Program Download

The example for this section is '**rosrobotcontrollerm4_app_ctl**'.

4. APP Usage

The car supports Bluetooth control via the app, allowing you to control its movement in three modes: button remote control, gravity sensing, and joystick remote control. The movement speed is measured in mm/s (millimeters per second). You can control the speed of the car chassis, and each press of the acceleration or deceleration button increases or decreases the speed by 50 mm/s. Additionally, you can control the rotation and centering of the pan-tilt on the car.

4.1 Bluetooth App Use Instructions

1. Bluetooth App Interface Introduction

The **top left corner** provides a detailed introduction to the app.

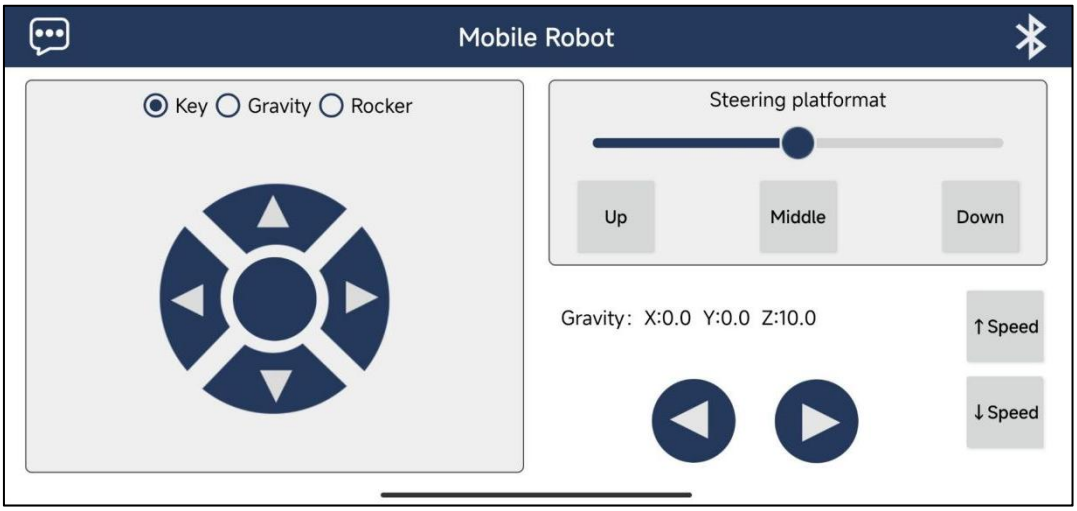
connected, the Bluetooth icon flashes, and the battery voltage is not displayed.

The **left section** is for selecting the car's movement control mode and control interface.

The **top right section** is for controlling the two-degree-of-freedom gimbal. The joystick controls the **left and right rotation** of the gimbal, while the buttons control the gimbal's tilt and centering functions.

The **middle right section** displays the phone's gravity sensor values for monitoring the car's gravity sensing mode.

The **bottom right section** has the acceleration and deceleration buttons for the car. Each press increases or decreases the car's speed by 50 mm/s.

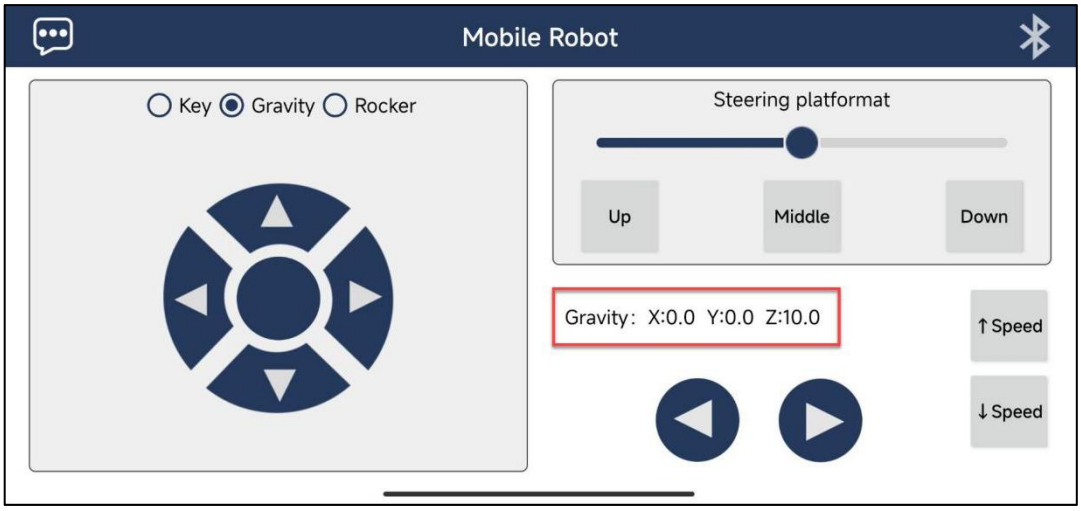


The app offers three control modes: button mode, gravity sensing mode, and joystick mode:

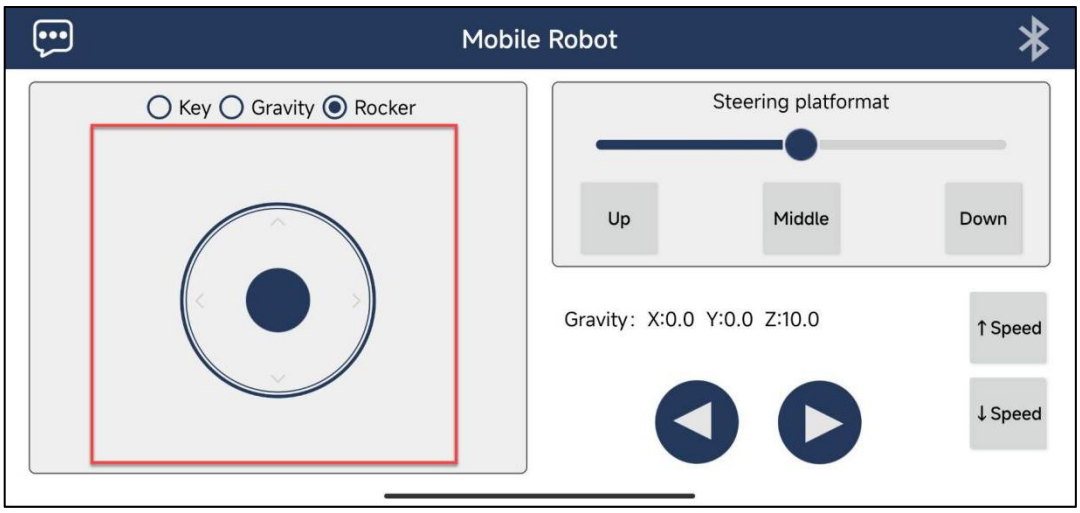
[1] Button Mode: This mode allows the car to move forward, backward, rotate left, turn left forward, rotate right, turn right forward, stop, move left, and move right (refer to the previous diagram).

[2] Gravity Sensing Mode: The movements are the same as in button mode, but the car is controlled by tilting the phone. Focus on the gravity sensor values. When the X-axis value is less than -3, the car moves forward; when it

is greater than 3, the car moves backward. When the Y-axis value is less than -3, the car moves left; when it is greater than 3, the car moves right.



[3] Joystick Mode: This mode adds two more movements compared to button mode, allowing for upper left, upper right, lower left, and lower right turns, corresponding to wide turns in each direction.



Whether the car can perform a spinning motion depends on its structure. Mecanum wheel cars and tracked cars can spin, while Ackermann steering cars and four-wheel differential cars can only turn. Only Mecanum wheel cars can perform lateral movements; the other cars will not respond to the lateral movement buttons.

2. **Turn on the car's power, and then turn on Bluetooth and the app on your phone.**

Important:

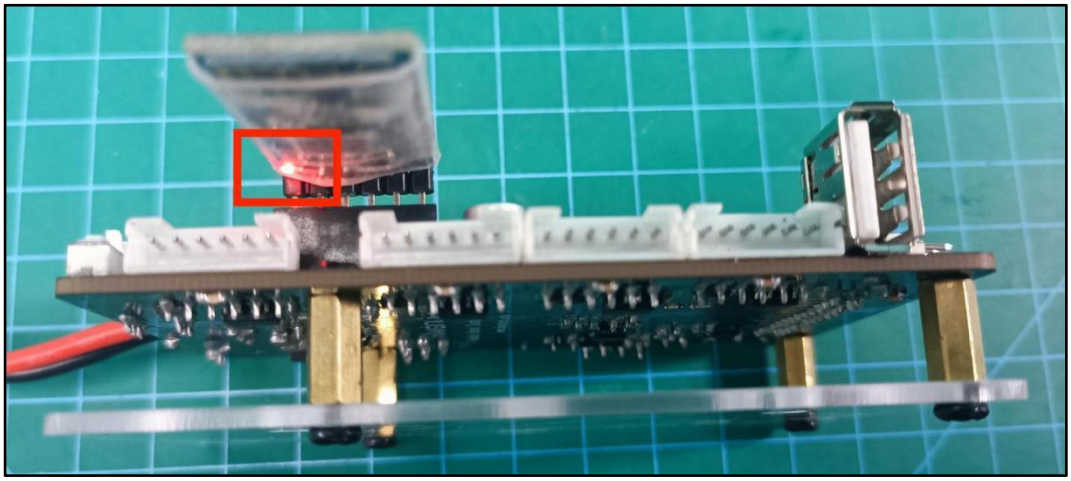
Please ensure the Bluetooth module is properly connected to the RRC control board before powering it on. This is necessary for the RRC to correctly initialize its Bluetooth functionality.

If the Bluetooth module is connected after the board is powered on, the RRC will not emit a beep upon successful Bluetooth connection, and the mobile app will not be able to control the RRC as expected.

If you must connect the Bluetooth module after powering on, please press the onboard RST button to reset the program and enable normal operation.

When not connected, the light on the Bluetooth module will flash.

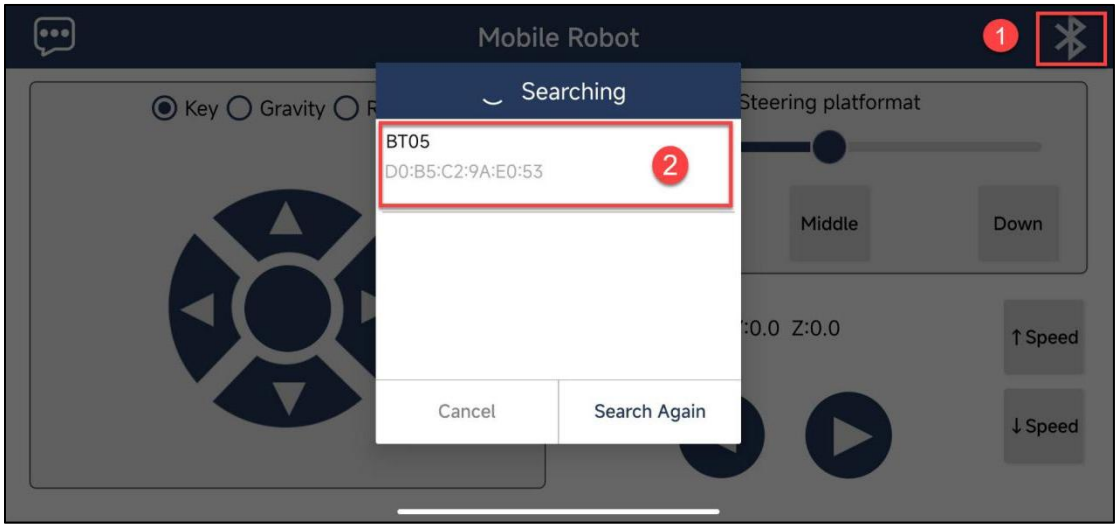
Once the car's Bluetooth is connected, the indicator light on the Bluetooth module will remain solid, and the buzzer will beep once.



3. **Click on the Bluetooth icon in the top right corner of the app to search for and connect to the car's Bluetooth, enabling control of the car's movement.**

Turn on Bluetooth on your phone. Open the app and click on the Bluetooth

icon in the top right corner. Once you find the car's Bluetooth, click to connect. Once connected, the indicator light on the car's Bluetooth module will remain solid, and the car's battery voltage will be displayed in the top right corner of the app.



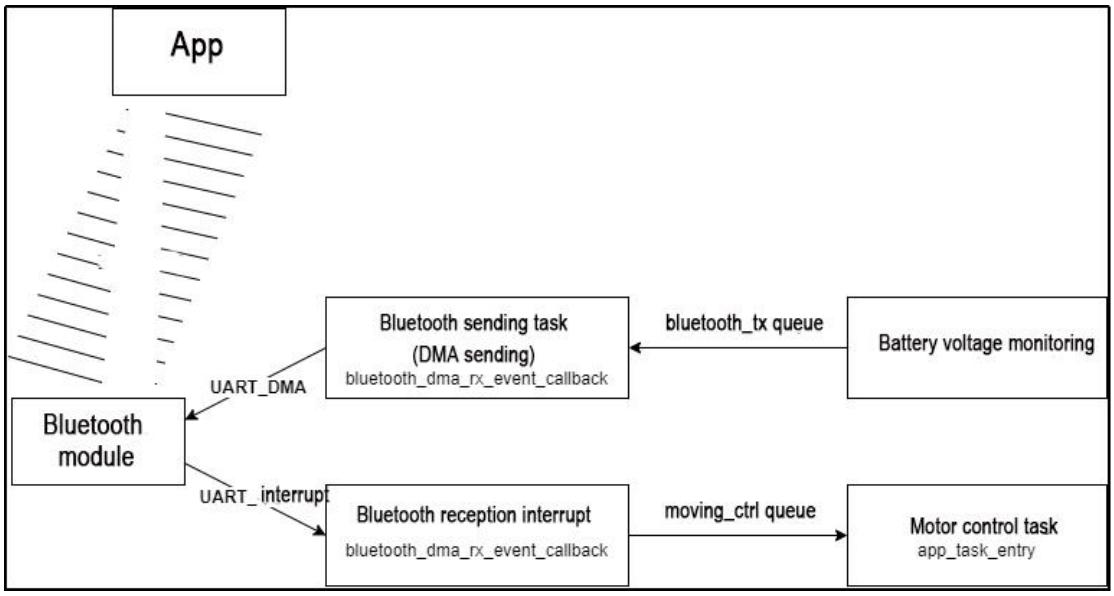
5. PIN Info Configuration

By checking the pin information using STM32CubeMX software, you can find that the communication between the Bluetooth module and the microcontroller is through UART communication. The corresponding chip pins are: PD6 and PD5.



6. Source Code Analysis

4.2 Control Diagram



As shown in the diagram above, the process is as follows:

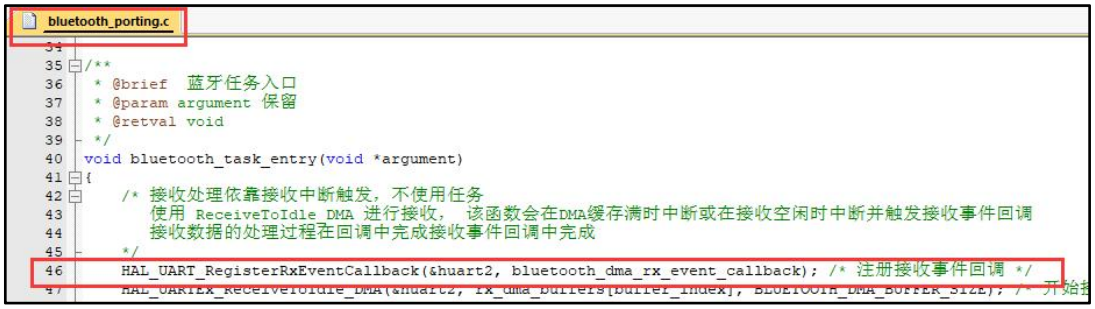
The mobile app sends signals to the Bluetooth module, which communicates with the microcontroller via UART.

When the Bluetooth receives information, it triggers a Bluetooth interrupt, receives the Bluetooth information, and sends it to the motor control task through an RTOS queue, thereby controlling the car's movement.

After the battery voltage is periodically detected, the voltage data is sent to the Bluetooth transmission task through a queue. The Bluetooth transmission task sends the data to the Bluetooth module via UART, and then the Bluetooth module sends the information to the mobile app, displaying it in the top right corner of the app.

4.3 Code Analysis

1、Analysis of the Code for UART Interrupt Reception of Bluetooth Module Data:



In the 'bluetooth_porting.c' file under the 'Hiwonder/Porting' project folder, the Bluetooth receive interrupt event callback function is registered.



In the 'bluetooth_porting.c' file, the 'bluetooth_dma_rx_event_callback()' callback function sends the information received from the Bluetooth module through the 'moving_ctrl' queue to the motor control task.

2、The motor control task receives and controls the motor movement.

app.c

```
36
37  /* 用户入口函数 */
38  void app_task_entry(void *argument)
39  {
40      /* 声明外部句柄 */
41      //蜂鸣器句柄
42      extern osTimerId_t buzzer_timerHandle;
43      //电量监控句柄
44      extern osTimerId_t battery_check_timerHandle;
45
46      //运动控制的队列句柄
47      //手柄的信号是在 gampad_handle.c 中USBH_HID_EventCallback()回调函数中压入
48      //在用户入口函数中取出控制小车运动
49      extern osMessageQueueId_t moving_ctrl_queueHandle;
50
51      /* 硬件初始化 */
52      motors_init();      //电机初始化
53      pwm_servos_init();  //云台舵机初始化
54      buzzers_init();     //蜂鸣器初始化
55      //开启蜂鸣器定时器，让其在中断中运作，后面调用接口函数即可
56      //参数1：定时器句柄 ，  参数2：定时器的工作间隔 ms
57      osTimerStart(buzzer_timerHandle, BUZZER_TASK_PERIOD);
58      //开启电量健康定时器，实时监控电量
59      osTimerStart(battery_check_timerHandle, BATTERY_TASK_PERIOD);
60
61      char msg = '\0';
62      uint8_t msg_prio;
63      //初始化 运动控制的队列（参数：队列句柄）
64      osMessageQueueReset(moving_ctrl_queueHandle);
65
66      //初始化底盘电机运动参数
67      chassis_init();
68
69      //选择底盘类型
70      // ! ! !
71      // 注意：四轮差速车底盘 使用 CHASSIS_TYPE_TI4WD  ! ! !
72      // ! ! !
73      set_chassis_type(CHASSIS_TYPE_TI4WD);
74
```

The 'app_task_entry()' function in the 'app.c' file under the 'Hiwonder/System' project folder is the entry function for the motor control task.

Within the for loop, it waits for information from the 'moving_ctrl' queue. If there is information available, it passes it to the chassis control function to control the movement of the car.

```
75      // 循环  : RTOS任务中的循环，必须要有osDelay或者其他系统阻塞函数，否则会导致系统异常
76      for(;;) {
77
78          //接收 运动控制队列 中的信息，若获取超100ms，则视为不成功，并使电机停止，跳过 这次循环
79          //osMessageQueueGet() 取出队列中的消息
80          // 参数1 : 消息队列句柄
81          // 参数2 : 待放入对象的地址 （这里为char类型地址）
82          // 参数3 : 消息优先级
83          // 参数4 : 超时设定（可设置等待获取时间）
84          if(osMessageQueueGet(moving_ctrl_queueHandle, &msg, &msg_prio, 100) != osOK) {
85              chassis->stop(chassis);
86              continue;
87          }
88
89          printf("msg: %c\r\n", msg); //debug打印
90
91          ti4wd_control(msg);      //底盘控制函数
92
93      }
94  }
95
```

3、Battery Voltage Timed Monitoring

In the 'app_task_entry()' function in the 'app.c' file under the 'Hiwonder/System' project folder, a timer is started to check the voltage every 50ms.

app.c

```
56      //参数1：定时器句柄 ，  参数2：定时器的工作间隔 ms
57      osTimerStart(buzzer_timerHandle, BUZZER_TASK_PERIOD);
58      //开启电量健康定时器，实时监控电量
59      osTimerStart(battery_check_timerHandle, BATTERY_TASK_PERIOD);
```

The 'battery_check_timer_callback()' function in the 'battery_handle.c' file under the 'Hiwonder/System' project folder is the callback function for the timer. It runs every time the 50ms interval is reached.

This function first checks if the ADC sampling value is correct. If it is, it converts it into a battery voltage value and continues sampling. Then, it checks if the number of detections has reached (1 * 1000 / BATTERY_TASK_PERIOD) times (i.e., 200 times, totaling 1 second). When the time is up, it sends the battery data once to the 'bluetooth_tx' queue."

battery_handle.c

```
13
14  void battery_check_timer_callback(void *argument)
15  {
16      if(adc_value[0] != 0 && adc_value[0] != 4095) {
17          /* 1210.0是内部参考电压值，  100k + 10k 电阻分压，  实际电压是测量电压的11倍 */
18          float volt = 1210.0f / ((float)adc_value[0]) * ((float)adc_value[1]) * 11.0f ;
19          volt = volt > 20000 ? 0 : volt;
20          battery_volt = battery_volt == 0 ? volt : battery_volt * 0.95f + volt * 0.05f;
21      }
22      printf("%d\r\n", adc_value[0]);
23      HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_value, 2);
24
25  #if  ENABLE_BLUETOOTH&&ENABLE_BLUETOOTH_BATTERY_REPORT
26      static int battery_report_count = 0;
27      battery_report_count++;
28      if(battery_report_count > (int)(1 * 1000 / BATTERY_TASK_PERIOD)) { /* 定时发送蓝牙电压报告 */
29          battery_report_count = 0;
30          char msg[8];
31          sprintf(&msg[1], "V%dV", (int)(battery_volt + 0.5f)); /* 组织蓝牙电量消息 */
32          printf("%s\r\n", &msg[1]);
33          msg[0] = strlen(&msg[1]);
34          osMessageQueuePut(bluetooth_tx_queueHandle, msg, 0, 0); /* 压入发送队列 */
35      }
36  #endif
```

4、Bluetooth Sending Task

In the 'bluetooth_porting.c' file under the 'Hiwonder/Porting' project folder, the 'bluetooth_task_entry()' function is the entry function for the Bluetooth transmission task.

Within the for loop, it waits for idle signals (time when the microcontroller is not processing other tasks). If there is idle time, it retrieves the battery data from the 'bluetooth_tx' queue. Then, it checks if the UART is ready. If it is, it sends the battery message to the Bluetooth module.

bluetooth_porting.c

```
/**
 * @brief  蓝牙任务入口
 * @param argument 保留
 * @retval void
 */
void bluetooth_task_entry(void *argument)
{
    /* 接收处理依靠接收中断触发，不使用任务
       使用 ReceiveToIdle_DMA 进行接收，该函数会在DMA缓存满时中断或在接收空闲时中断并触发接收事件回调
       接收数据的处理过程在回调中完成接收事件回调中完成
    */
    HAL_UART_RegisterRxEventCallback(&huart2, bluetooth_dma_rx_event_callback); /* 注册接收事件回调 */
    HAL_UARTEx_ReceiveToIdle_DMA(&huart2, rx_dma_buffers[buffer_index], BLUETOOTH_DMA_BUFFER_SIZE); /* 开始接收 */

    /* 发送处理，通过队列触发发送 */
    uint8_t tx_buffer[8];
    for(;;) {
        osSemaphoreAcquire(bluetooth_tx_idleHandle, osWaitForever); /* 等待发送空闲信号 */
        /* 从发送队列中取出数据 */
        osStatus_t status = osMessageQueueGet(bluetooth_tx_queueHandle, tx_buffer, NULL, osWaitForever);
        if(osOK == status) {
             /* 注册 DMA 接收完成回调 */
            HAL_UART_RegisterCallback(&huart2, HAL_UART_TX_COMPLETE_CB_ID, bluetooth_dma_tx_complete_callback);
            /* 消息的第一个字节作为要发送的数据的长度，第二字节开始为要发送的数据 */
            HAL_UART_Transmit_DMA(&huart2, &tx_buffer[1], tx_buffer[0] > 7 ? 7 : tx_buffer[0]);  /* 触发 DMA 发送*
        }
    }
}
```

Appendix:

The Bluetooth Secondary Development Control Protocol Table is as follows:

Character	Contained Info
A	Go forward
B	Turn left with a wide curve
C	Turn left
D	Turn left with a wide curve to the rear left
E	Reverse
F	Turn right with a wide curve to the rear right
G	Turn right
H	Turn right with a wide curve to the front right
I	Chassis stop
J	Pan-tilt downward rotation
K	
L	Pan-tilt rotation to the left
M	
N	Pan-tilt rotation upwards

O	
P	Pan-tilt rotation to the right
Q	
R	Pan-tilt reset
f	One short beep from the buzzer
S	Speed up by one level
j	Slow down by one level
n	Left lateral movement (only available for Mecanum wheel chassis)
l	Right lateral movement (only available for Mecanum wheel chassis)
p	Phone receives battery voltage, with the character 'V' as a delimiter, and the battery voltage string in between
VxxxV	Pan-tilt reset

Examples:

1. Need continuous forward movement of the car chassis: The phone Bluetooth sends the character 'A' to the Bluetooth module at a frequency of 10 times per second. When the transmission stops, the car chassis stops moving.
2. Need the buzzer to beep once: The phone Bluetooth sends the character 'S' to the Bluetooth module, and the buzzer beeps once.
3. Need the pan-tilt to rotate left: The phone Bluetooth sends the character 'L' to the Bluetooth module at a frequency of 10 times per second. When the transmission stops, the pan-tilt stops rotating.
4. Need to increase the speed of the car chassis by one level: The phone Bluetooth sends the character 'j' to the Bluetooth module, and the speed of the car chassis increases by one level.
5. Phone Bluetooth receives the battery voltage of the car chassis: If the phone Bluetooth receives the string "V7400V", it means the current battery voltage is 7400mV, or 7.4V.